Namespace Crystal.Avalonia

Classes

CrystalApplication

The base application class, extending Avalonia's Avalonia.Application. Provides the foundation for modular development and MVVM binding.

CrystalOptions

Provides global configuration options for the Crystal.Avalonia framework.

ModuleManager

Manages and initializes application modules. Automatically created and used by the framework during OnFrameworkInitializationCompleted().

MvvmManager

Manages View-to-ViewModel mapping and MVVM binding registration.

ViewModelLocator

Provides automatic ViewModel binding via an attached property. Works in conjunction with EnableViewModelLocator.

Interfaces

IModule

Defines an application module, which is a self-contained unit of functionality. Modules allow you to organize your application into independent feature areas, each managing its own services and views.

IModuleRegistrar

Provides module registration functionality. Used to register all functional modules during application startup.

Class CrystalApplication

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

The base application class, extending Avalonia's Avalonia.Application. Provides the foundation for modular development and MVVM binding.

```
public class CrystalApplication : Application, INotifyPropertyChanged, IDataContextProvider,
IGlobalDataTemplates, IDataTemplateHost, IGlobalStyles, IStyleHost, IThemeVariantHost,
IResourceHost, IResourceNode, IApplicationPlatformEvents
```

Inheritance

object ← AvaloniaObject ← Application ← CrystalApplication

Implements

INotifyPropertyChanged, IDataContextProvider, IGlobalDataTemplates, IDataTemplateHost, IGlobalStyles, IStyleHost, IThemeVariantHost, IResourceHost, IResourceNode, IApplicationPlatformEvents

Inherited Members

Application.DataContextProperty , Application.ActualThemeVariantProperty , Application.RequestedThemeVariantProperty , Application.NameProperty , Application.Initialize() , Application.TryGetResource(object, ThemeVariant, out object) , Application.RegisterServices() , Application.OnPropertyChanged(AvaloniaPropertyChangedEventArgs) , Application.DataContext , Application.RequestedThemeVariant , Application.ActualThemeVariant , Application.Current , Application.DataTemplates , Application.Resources , Application.Styles , Application.ApplicationLifetime , Application.PlatformSettings , Application.Name , Application.ResourcesChanged , Application.UrlsOpened , Application.ActualThemeVariantChanged , AvaloniaObject.CheckAccess() , AvaloniaObject.VerifyAccess() , AvaloniaObject.ClearValue(AvaloniaProperty) , AvaloniaObject.ClearValue<T>(AvaloniaProperty<T>) , AvaloniaObject.ClearValue<T>(StyledProperty<T>) , AvaloniaObject.ClearValue<T>(DirectPropertyBase<T>) , AvaloniaObject.Equals(object) , AvaloniaObject.GetHashCode() , AvaloniaObject.GetValue(AvaloniaProperty) , AvaloniaObject.GetValue<T>(StyledProperty<T>) , AvaloniaObject.GetValue<T>(DirectPropertyBase<T>) , AvaloniaObject.GetBaseValue<T>(StyledProperty<T>) ,

AvaloniaObject.IsAnimating(AvaloniaProperty) , AvaloniaObject.IsSet(AvaloniaProperty) , AvaloniaObject.SetValue(AvaloniaProperty, object, BindingPriority) , AvaloniaObject.SetValue<T>(StyledProperty<T>, T, BindingPriority) , AvaloniaObject.SetValue<T>(DirectPropertyBase<T>, T) , AvaloniaObject.SetCurrentValue(AvaloniaProperty, object) , AvaloniaObject.SetCurrentValue<T>(StyledProperty<T>, T) , AvaloniaObject.Bind(AvaloniaProperty, IObservable<object>, BindingPriority) , AvaloniaObject.Bind<T>(StyledProperty<T>, IObservable<object>, BindingPriority) , AvaloniaObject.Bind<T>(StyledProperty<T>, IObservable<T>, BindingPriority) , AvaloniaObject.Bind<T>(StyledProperty<T>, IObservable<BindingValue<T>>, BindingPriority) , AvaloniaObject.Bind<T>(DirectPropertyBase<T>, IObservable<object>) , AvaloniaObject.Bind<T>(DirectPropertyBase<T>, IObservable<T>) , AvaloniaObject.Bind<T>(DirectPropertyBase<T>, IObservable<BindingValue<T>>) , AvaloniaObject.CoerceValue(AvaloniaProperty) , AvaloniaObject.UpdateDataValidation(AvaloniaProperty, BindingValueType, Exception) , AvaloniaObject.OnPropertyChangedCore(AvaloniaPropertyChangedEventArgs) , AvaloniaObject.RaisePropertyChanged<T>(DirectPropertyBase<T>, T, T) , AvaloniaObject.SetAndRaise<T>(DirectPropertyBase<T>, ref T, T) , AvaloniaObject.InheritanceParent , AvaloniaObject.this[AvaloniaProperty] , AvaloniaObject.this[IndexerDescriptor] , AvaloniaObject.PropertyChanged , object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Examples

```
public class MyApp : CrystalApplication
{
    public override void RegisterModules(IModuleRegistrar moduleRegistrar)
    {
        moduleRegistrar.RegisterModule<MyModule>();
    }

    public override void RegisterServices(IServiceCollection services)
    {
        services.AddMvvmBindingTransient<MainView, MainViewModel>();
    }

    public override void CreateShell(IServiceProvider serviceProvider)
    {
        if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop)
            desktop.MainWindow = new MainWindow();
```

```
    }
}
```

Remarks

Usage steps:

1. Define modules: Create classes implementing IModule.
2. Override RegisterModules(IModuleRegistrar): Register all functional modules.
3. Override RegisterServices(IServiceCollection): Register application-level services (optional).
4. Override CreateShell(IServiceProvider): Create the main window or main view.

Methods

CreateShell(IServiceProvider)

Override this method to create the main shell (window or view) of the application.

```
public virtual void CreateShell(IServiceProvider serviceProvider)
```

Parameters

`serviceProvider` IServiceProvider

The service provider, can be used to resolve services registered in RegisterServices(IServiceCollection) or modules.

OnFrameworkInitializationCompleted()

```
public override void OnFrameworkInitializationCompleted()
```

RegisterModules(IModuleRegistrar)

Override this method to register all functional modules for the application.

```
public virtual void RegisterModules(IModuleRegistrar moduleRegistrar)
```

Parameters

`moduleRegistrar` IModuleRegistrar

The module registrar, used to register IModule implementations.

Examples

```
public override void RegisterModules(IModuleRegistrar moduleRegistrar)
{
    moduleRegistrar.RegisterModule<HomeModule>();
    moduleRegistrar.RegisterModule<SettingsModule>();
}
```

RegisterServices(IServiceCollection)

Override this method to register application-level services. It is recommended to use AddMvvmBindingTransient<TView, TViewModel>(IServiceCollection) or AddMvvmBindingSingleton<TView, TViewModel>(IServiceCollection) to register View/ViewModel pairs.

```
public virtual void RegisterServices(IServiceCollection services)
```

Parameters

`services` IServiceCollection

The service collection to add application-level services to.

Class CrystalOptions

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Provides global configuration options for the Crystal.Avalonia framework.

```
public static class CrystalOptions
```

Inheritance

object ← CrystalOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

EnableViewModelLocator

Gets or sets whether the ViewModelLocator is enabled. When enabled, the system automatically resolves the corresponding ViewModel based on the View type and injects it into the DataContext. Defaults to `true`.

```
public static bool EnableViewModelLocator { get; set; }
```

Property Value

bool

Remarks

When enabled, the AutoWireViewModelProperty attached property takes effect. You can set `vm:ViewModelLocator.AutoWireViewModel="True"` in XAML to enable auto-binding.

Interface IModule

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Defines an application module, which is a self-contained unit of functionality. Modules allow you to organize your application into independent feature areas, each managing its own services and views.

```
public interface IModule
```

Methods

InitializeModule(IServiceProvider)

Override this method to perform module initialization logic. At this point, all services have been registered and can be resolved from `serviceProvider`.

```
void InitializeModule(IServiceProvider serviceProvider)
```

Parameters

`serviceProvider` IServiceProvider

The service provider, can be used to resolve services registered in this module.

RegisterServices(IServiceCollection)

Override this method to register services that belong to this module. It is recommended to use AddMvvmBindingTransient<TView, TViewModel>(IServiceCollection) or AddMvvmBindingSingleton<TView, TViewModel>(IServiceCollection) to register View/ViewModel pairs.

```
void RegisterServices(IServiceCollection services)
```

Parameters

`services` IServiceCollection

The service collection to register services into.

Interface IModuleRegistrar

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Provides module registration functionality. Used to register all functional modules during application startup.

```
public interface IModuleRegistrar
```

Methods

RegisterModule(IModule)

Registers an already instantiated module.

```
void RegisterModule(IModule module)
```

Parameters

`module` IModule

The module instance to register.

RegisterModule<TModule>()

Registers a module type. The system will automatically instantiate the module.

```
void RegisterModule<TModule>() where TModule : IModule
```

Type Parameters

`TModule`

The module type, must implement IModule and have a public parameterless constructor.

Class ModuleManager

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Manages and initializes application modules. Automatically created and used by the framework during OnFrameworkInitializationCompleted().

```
public class ModuleManager : IModuleRegistrar
```

Inheritance

object ← ModuleManager

Implements

IModuleRegistrar

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

LoadModules

Gets the list of registered modules.

```
public IList<IModule> LoadModules { get; set; }
```

Property Value

IList<IModule>

Methods

InitModules(IServiceProvider)

Iterates through all registered modules and calls their InitializeModule(IServiceProvider) method to perform module initialization. Automatically invoked by the framework after the service provider is built.

```
public void InitModules(IServiceProvider serviceProvider)
```

Parameters

`serviceProvider` IServiceProvider

The service provider.

InitService(IServiceCollection)

Iterates through all registered modules and calls their RegisterServices(IServiceCollection) method to register services into the container. Automatically invoked by the framework before the service provider is built.

```
public void InitService(IServiceCollection services)
```

Parameters

`services` IServiceCollection

The service collection.

RegisterModule(IModule)

Registers an already instantiated module.

```
public void RegisterModule(IModule module)
```

Parameters

`module` IModule

The module instance to register.

RegisterModule<TModule>()

Registers a module type. The system will automatically instantiate the module.

```
public void RegisterModule<TModule>() where TModule : IModule
```

Type Parameters

`TModule`

The module type, must implement IModule and have a public parameterless constructor.

Class MvvmManager

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Manages View-to-ViewModel mapping and MVVM binding registration.

```
public static class MvvmManager
```

Inheritance

object ← MvvmManager

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

After registering View/ViewModel pairs using this class, the system will automatically:

- Inject the DataContext via ViewModelLocator when EnableViewModelLocator is enabled
- Resolve the corresponding View via Crystal.Avalonia.ViewLocator based on the ViewModel type

Properties

ServiceProvider

Gets the current application's service provider instance. Available after OnFrameworkInitializationCompleted() completes.

```
public static IServiceProvider? ServiceProvider { get; set; }
```

Property Value

IServiceProvider

Methods

AddMvvmBindingSingleton<TView, TViewModel>(IServiceCollection)

Registers a View and ViewModel pair with the View as Transient and ViewModel as Singleton.

```
public static void AddMvvmBindingSingleton<TView, TViewModel>(this IServiceCollection
services) where TView : Control where TViewModel : class
```

Parameters

`services` IServiceCollection

The service collection.

Type Parameters

`TView`

The View type, must inherit from Avalonia.Controls.Control.

`TViewModel`

The ViewModel type, can be any class.

Examples

```
services.AddMvvmBindingSingleton<SettingsView, SettingsViewModel>();
```

AddMvvmBindingTransient<TView, TViewModel>(IServiceCollection)

Registers a View and ViewModel pair as Transient mappings. Both the View and ViewModel are registered as Transient in the DI container.

```
public static void AddMvvmBindingTransient<TView, TViewModel>(this IServiceCollection
services) where TView : Control where TViewModel : class
```

Parameters

`services` IServiceCollection

The service collection.

Type Parameters

`TView`

The View type, must inherit from Avalonia.Controls.Control.

`TViewModel`

The ViewModel type, can be any class.

Examples

```
services.AddMvvmBindingTransient<MainView, MainViewModel>();
```

GetViewType(Type)

Looks up the View type corresponding to the given ViewModel type.

```
public static Type? GetViewType(Type vmType)
```

Parameters

`vmType` Type

The ViewModel type.

Returns

Type

The corresponding View type, or `null` if not found.

GetVmType(Type)

Looks up the ViewModel type corresponding to the given View type.

```
public static Type? GetVmType(Type viewType)
```

Parameters

`viewType` Type

The View type.

Returns

Type

The corresponding ViewModel type, or `null` if not found.

Class ViewModelLocator

Namespace: Crystal.Avalonia

Assembly: Crystal.Avalonia.dll

Provides automatic ViewModel binding via an attached property. Works in conjunction with EnableViewModelLocator.

```
public class ViewModelLocator
```

Inheritance

object ← ViewModelLocator

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Usage in XAML:

```
<Window xmlns:vm="using:Crystal.Avalonia"
        vm:ViewModelLocator.AutoWireViewModel="True">
```

When `AutoWireViewModel` is set to `True`, the system automatically resolves the corresponding ViewModel from the DI container and assigns it to the control's `DataContext`.

Fields

AutoWireViewModelProperty

Identifies the AutoWireViewModelProperty attached property registration. When set to `True`, the system automatically binds the corresponding ViewModel to the control's DataContext.

```
public static readonly AttachedProperty<bool> AutoWireViewModelProperty
```

Field Value

AttachedProperty<bool>

Methods

GetAutoWireViewModel(Control)

Gets the AutoWireViewModel attached property value for the specified control.

```
public static bool GetAutoWireViewModel(Control element)
```

Parameters

`element` Control

The control to query.

Returns

bool

`True` if the control has automatic ViewModel binding enabled.

SetAutoWireViewModel(Control, bool?)

Sets the AutoWireViewModel attached property value for the specified control.

```
public static void SetAutoWireViewModel(Control element, bool? value)
```

Parameters

`element` Control

The control to set.

`value` bool?

Whether to enable automatic binding.